

MedhasAcademy Task

Full Stack Developer Assignment — Journaling Web Application

1. Project Overview

Build a simple full stack journaling application. A user should be able to create an account, sign in, write journal entries (with a Title and Description), save them, and view all their saved journals in a list. The focus of this task is **working functionality**, not visual design. Keep the UI plain and minimal — a clean, functional layout is enough.

2. Tech Stack

Layer	Technology
Framework	Next.js (single project for both frontend and backend)
Backend	Next.js API Routes (no separate Express/Node server)
Frontend	Next.js pages/components (React)
Database	MongoDB (MongoDB Atlas recommended for a free hosted DB)
Auth	JWT-based authentication (token saved on the client)
Deployment	Vercel or Render (any one, must produce a live link)
Version Control	GitHub (public or shared repository)

Note: This must be built purely in the Next.js environment — both the frontend pages and the backend API logic live inside the same Next.js project (using the app/pages API routes). Do not create a separate backend server/repo.

3. Required Routes / Pages

Route	Purpose
/register	New user signs up (name/email, password, etc.)
/sign-in	Existing user logs in with email and password
/create-journal	Authenticated user writes a new journal (Title + Description) and saves it
/list-journals	Shows all saved journals of the logged-in user, sorted newest first (by date, descending)

4. Authentication & App Flow

- When a new visitor opens the site, if they have no valid saved token, they should be able to reach /register or /sign-in.
- On successful sign in or registration, the server should return a JWT token. Save this token on the client (e.g. localStorage/cookie).

- Once logged in, the token should be sent with each request that needs authentication (e.g. in the Authorization header) so the backend knows who the user is.
- If a user already has a valid saved token (i.e. already logged in) and opens the website again, the first/landing route should automatically take them directly to /list-journals instead of showing the sign-in page again.
- On the /list-journals page, include a clearly visible button (e.g. "+ New Journal") that takes the user to /create-journal.
- After a journal is created on /create-journal, save it to MongoDB tied to that user's account, and redirect back to /list-journals so the new entry appears at the top.
- Journals on /list-journals must be displayed in descending order of date (most recently created/updated journal appears first).
- Add a simple logout option that clears the token and sends the user back to /sign-in.

5. Basic Data Model (suggested, candidate may adjust)

User

name, email, password (hashed), createdAt

Journal

title, description, userId (reference to the owner), createdAt

6. Functional Requirements (priority order)

- 1 User registration with basic validation (no duplicate emails, password should be stored securely/hashed).
- 2 User sign in with email + password, returns a JWT token on success.
- 3 Protected routes: /create-journal and /list-journals should only be accessible to a logged-in user (redirect to /sign-in otherwise).
- 4 Create journal entry with Title and Description fields, saved against the logged-in user.
- 5 List all journal entries of the logged-in user, sorted by date descending.
- 6 Auto-redirect logic: logged-in users land directly on /list-journals; logged-out users land on /sign-in (or /register).

7. What is NOT expected

- No need for a polished or fancy UI — plain, simple, functional pages are enough.
- No need for extra features like editing/deleting journals, categories, tags, image uploads, etc. (candidate may add these only if time permits, but they are not required).
- No need for password reset / forgot-password flow.
- No need for a separate backend framework (Express, NestJS, etc.) — use Next.js API routes only.

8. Deliverables

- Complete source code pushed to a GitHub repository (public, or shared with access).

- A live working link — deployed on Vercel or Render (either is fine).
- MongoDB used as the database (Atlas free tier is fine for this).
- A proper README.md in the repository explaining, in simple language, what the project does, the tech stack used, how to run it locally, and the live link.

9. Timeline

Deadline: 2 days from the date this task is shared. Functionality and correct working flow are the top priority — not UI polish. A simple, fully working app is far better than a good-looking but broken/incomplete one.

10. Doubts / Questions

If there are any doubts along the way — including help with setting up the project, MongoDB connection, or deployment — feel free to ask before or during the task.

Task assigned by: **Aditya Joshi**